

tarea 1:

RETOs ROMPE-CÓDIGO

reto 1

lenguaje natural

1. Definir variables:
vaso A = jugo, vaso B = agua, vaso C = vacío.
2. Cojo el vaso A.
3. Vierto el contenido del vaso A en el vaso C.
4. Suelto el vaso A (ahora está vacío).
5. Actualizo variable: vaso C = jugo, vaso A = vacío.
6. Cojo el vaso B.
7. Vierto el contenido del vaso B en el vaso A.
8. Suelto el vaso B (ahora está vacío).
9. Actualizo variable: vaso A = agua, vaso B = vacío.
10. Cojo el vaso C.
11. Vierto el contenido del vaso C en el vaso B.
12. Suelto el vaso C (ahora está vacío).
13. Actualizo variable: vaso B = jugo, vaso C = vacío.
14. Verifico: vaso A = agua, vaso B = jugo, vaso C = vacío

reto 2

lenguaje natural

1. Definir variables: edad, tiene urgencia, minutos espera, prioridad.
2. Tomo el dato de la edad del paciente.
3. Pregunto: ¿edad es menor a 18?
4. Si sí, marco camino "menor de edad" y continúo en el paso 5.
5. Pregunto (menor de edad): ¿tiene urgencia?
6. Si tiene urgencia, asigno prioridad = Alta y voy al paso 17.
7. Si no tiene urgencia, continúo en el paso 8.
8. Pregunto (menor sin urgencia): ¿espera es mayor o igual a 15 minutos?
9. Si sí, asigno prioridad = Alta y voy al paso 17.
10. Si no, asigno prioridad = Media y voy al paso 17.
11. Si en el paso 3 la respuesta fue "no", pregunto: ¿edad es mayor a 60?
12. Si sí, marco camino "adulto mayor" y continúo en el paso 13.
13. Pregunto (adulto mayor): ¿tiene urgencia?
14. Si tiene urgencia, asigno prioridad = Alta y voy al paso 17.
15. Si no tiene urgencia, pregunto: ¿espera es mayor o igual a 30 minutos?
- Si sí, asigno prioridad = Alta y voy al paso 17.
- Si no, asigno prioridad = Media y voy al paso 17.
16. Si en el paso 11 la respuesta fue "no" (edad entre 18 y 60), marco camino "adulto" y pregunto: ¿tiene urgencia?
- Si tiene urgencia, asigno prioridad = Alta y voy al paso 17.
- Si no tiene urgencia, pregunto: ¿espera es mayor o igual a 60 minutos?
- Si sí, asigno prioridad = Media y voy al paso 17.
- Si no, asigno prioridad = Baja y voy al paso 17.
17. Muestro el resultado: la prioridad final asignada al paciente.
18. Repito todo el proceso para el siguiente paciente.

tarea 1:

RETOs ROMPE-CÓDIGO

Reto 3

lenguaje natural

1. Definir variables: número actual = 50, salto = 25, intento = 0.
2. sumo 1 al contador de intentos.
3. Digo en voz alta: "¿ES el número [número actual]?"
4. Escucho la respuesta del otro jugador.
5. si la respuesta es "correcto", termino el juego y anuncio el intento en que acerté.
6. si la respuesta es "mayor", sumo el salto al número actual.
7. si la respuesta es "menor", resto el salto al número actual.
8. Divido el salto entre 2 (lo reduzco a la mitad).
9. si el salto queda en 0, lo fijo en 1.
10. vuelvo al paso 2 con el nuevo número actual y el nuevo salto.
11. si llego a 7 intentos sin acertar, detengo el juego y anuncio que se agotaron los intentos.

Tarea 2

Técnicas de nomenclatura (case styles)

Estas convenciones se utilizan en programación para nombrar variables, funciones, clases y archivos, garantizando que el código sea legible y estandarizado.

camel case (Lower camel case):

Regla: La primera palabra comienza en minúscula y las palabras siguientes comienzan con mayúscula, sin espacios ni símbolos.

Ejemplo: `nombreusuario`, `calcularsalarioNeto`, `idEmpleado`.

Uso común: variables y métodos en JavaScript, Java y C#.

Pascal case (Upper camel case):

Regla: Todas las palabras comienzan con mayúscula, incluyendo la primera.

Ejemplo: `Nombreusuario`, `Calcularsalario`, `IdEmpleado`.

Uso común: Nombres de clases y componentes en la mayoría de los lenguajes de programación.

snake case:

Regla: Todas las palabras van en minúsculas y se separan mediante un guion bajo (`_`).

Ejemplo: `nombre_usuario`, `calcular_salario_neto`.

Uso común: variables y funciones en Python, y nombres de columnas en bases de datos (SQL).

kebab case (Dash case):

Regla: Todas las palabras van en minúsculas y se separan por un guion medio (`-`).

Ejemplo: `nombre-usuario`, `boton-principal`.

Uso común: Direcciones URL y nombres de clases en hojas de estilo CSS.

screaming snake case:

Regla: Todas las palabras van en mayúsculas y separadas por guion bajo.

Ejemplo: `VALOR_MAXIMO`, `PI`, `CONFIG_BASE_DATOS`.

Uso común: Definición de constantes globales que no cambian su valor.

Tarea 2

Ciclo de vida del Desarrollo de software (SDLC)

El SDLC (Software Development Life cycle) es el proceso estructurado que se sigue para planificar, crear, probar y desplegar un sistema de información. consta de las siguientes fases secuenciales o iterativas:

Planificación y Viabilidad:

se definen el alcance del proyecto, los costos estimados, los plazos de entrega, los recursos necesarios y los riesgos potenciales.

Análisis de Requisitos:

se recopilan las necesidades del cliente o usuario final de forma detallada. El resultado se plasma en un documento formal de especificación de requerimientos.

Diseño de Arquitectura:

Los arquitectos de software definen la estructura técnica. se diseñan las interfaces de usuario (UI), los flujos de datos, los diagramas de bases de datos y la arquitectura del sistema.

Desarrollo (codificación):

Los programadores escriben el código fuente real del software utilizando las tecnologías y lenguajes previamente seleccionados.

Pruebas (QA / Testing):

se somete el software a rigurosas pruebas (unitarias, de integración, de rendimiento y de seguridad) para detectar fallos (bugs) y asegurar que cumpla los requisitos del cliente.

Despliegue (Implementación):

El software se lanza al entorno de producción para que los usuarios finales puedan empezar a utilizarlo. Puede ser un lanzamiento gradual o total.

Mantenimiento y Evolución:

Es la fase más larga. consiste en corregir errores remanentes reportados por usuarios, actualizar el sistema para adaptarlo a nuevos entornos y añadir nuevas funciones solicitadas.

Tarea 2

Documentación de código en Python y C++

La documentación interna ayuda a otros desarrolladores (y a ti mismo en el futuro) a entender la lógica, los parámetros y el propósito de cada sección de código.

En Python

Python promueve el uso de herramientas nativas muy potentes para documentar estructuras completas:

comentarios de línea: se utiliza el símbolo de almohadilla (#). sirven para explicar líneas específicas de código.

docstrings (cadenas de documentación): se colocan inmediatamente después de la definición de una función, clase o módulo, delimitados por tres comillas dobles ("""). son legibles por herramientas de documentación automática (como sphinx).

En C++

C++ utiliza los comentarios tradicionales de la sintaxis estilo C, y suele integrarse con herramientas externas como Doxygen para generar manuales técnicos automáticos.

comentarios de una línea: se utiliza la doble barra diagonal (//).

comentarios de múltiples líneas o bloques: se encierran entre /* y */.

comentarios de documentación (Estilo Doxygen): se utilizan tres barras (///) o bloques especiales (/** ... */) con etiquetas específicas como @param o @return para que los generadores de documentación entiendan la estructura del código.